



भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Indian Institute of Technology Hyderabad

Fraud Analytics CS6890

Assignment - 5

Finding Clusters using Node2Vec embedding

Team Members

CS24MTECH14013	Mohit Manoj Patil
CS24MTECH14010	Veeresh Shukla
CS24MTECH12018	Deeba Afridi

Contents

1	Problem Definition	2
2	Expected Outcome	2
3	Dataset Specification	2
	3.1 Data Source and Format	2
	3.2 Data Columns and Types	2
	3.3 Preprocessing and Aggregation	3
	3.4 Graph Characteristics	3
	3.5 Data Snapshot	3
4	Algorithm	3
	4.1 Graph Construction	3
	4.2 Node2Vec Embedding Generation	4
	4.3 K-Means Clustering	4
	4.4 Optimal K Selection (Elbow Method)	4
	4.5 Dimensionality Reduction (PCA)	5
5	Approach	5
	5.1 Data Preparation	5
	5.2 Graph Construction Implementation	5
	5.3 Node2Vec Implementation Details	5
	5.4 Clustering Execution	6
	5.5 Visualization Generation	6
6	Results and Analysis	6
	6.1 Summary of Outputs	6
	6.2 Clustering Results and Visualization	7
	6.3 Analysis and Interpretation	7
	6.4 Limitations and Future Work	8
	6.5 Conclusion	8

List of Figures

1	Snapshot of the input data structure in <code>Payments.xlsx</code>	3
2	PCA Visualization of Node Embeddings Clustered with K=4	7

1 Problem Definition

The primary problem is to identify and analyze community structures or clusters within a network derived from financial transaction data. Given a dataset of payments between accounts (Senders and Receivers), the goal is to group accounts (nodes) that exhibit similar connectivity patterns or belong to potentially related sub-networks. This involves representing the transaction data as a graph, learning meaningful feature representations (embeddings) for each account node that capture its role and neighborhood context within the graph, and then applying a clustering algorithm to group nodes based on these learned embeddings. The challenge lies in effectively capturing the complex relationships in the graph structure through embeddings and determining an appropriate number of clusters to reveal underlying patterns, which could be relevant for applications like fraud detection, community analysis, or understanding transaction flows.

2 Expected Outcome

The expected outcomes of this analysis are:

- **Graph Representation:** A directed, weighted graph where nodes represent accounts and weighted edges represent the aggregated transaction volume between them.
- **Node Embeddings:** Low-dimensional vector representations (embeddings) for each node in the graph, learned using the Node2Vec algorithm. These embeddings should ideally encode the structural similarity and neighborhood information of the nodes.
- **Optimal Cluster Count (K):** An estimation of the optimal number of clusters present in the data, determined using the Elbow method applied to K-Means clustering results on the node embeddings.
- **Cluster Assignment:** Each node (account) assigned to one of the determined clusters based on the K-Means algorithm applied to the embeddings.
- **Visualization:** A 2D visualization (using PCA) of the node embeddings, colored according to their assigned cluster. This plot should provide a visual intuition about the separation and density of the identified clusters.
- **Analysis:** An interpretation of the results, commenting on the identified clusters, the effectiveness of the chosen K, and the insights gained from the visualization, considering the limitations (e.g., variance explained by PCA).

3 Dataset Specification

3.1 Data Source and Format

The dataset used for this assignment is sourced from an Excel file named `payments.xlsx`. It contains records of individual financial transactions.

3.2 Data Columns and Types

The analysis specifically utilizes the following columns from the dataset:

- **Sender:** Identifier for the account initiating the payment. Treated as a string data type.
- **Receiver:** Identifier for the account receiving the payment. Treated as a string data type.
- **Amount:** The monetary value of the transaction. Treated as a floating-point numeric data type.

3.3 Preprocessing and Aggregation

Several preprocessing steps are applied to the raw data:

- **Loading:** The data is loaded using the pandas library’s `read_excel` function.
- **Type Conversion & Cleaning:** Sender and Receiver IDs are explicitly converted to strings. The Amount column is converted to a numeric type, with any non-numeric entries coerced to ‘NaN’. Rows containing ‘NaN’ in any of the essential columns (Sender, Receiver, Amount) are dropped.
- **Transaction Aggregation:** Since the goal is to model relationships between accounts, multiple transactions occurring between the same Sender and Receiver pair are aggregated. The weight of the edge connecting a Sender node to a Receiver node in the final graph represents the *sum* of all transaction amounts between them.
- **Handling Isolated Nodes:** After constructing the graph based on aggregated transactions, any nodes that have no incoming or outgoing edges (i.e., isolated nodes) are removed from the graph, as they do not participate in the transaction network relevant for Node2Vec.

3.4 Graph Characteristics

Based on the console output provided, the dataset and preprocessing steps result in the following graph structure:

- **Initial Transactions Processed:** 130,535.
- **Nodes (Accounts):** 799 unique nodes remaining after aggregation and removal of isolates.
- **Edges (Aggregated Transactions):** 5,358 directed, weighted edges connecting the nodes. The weight signifies the total transaction volume.

3.5 Data Snapshot

Figure 1 provides a visual sample of the input data format found in the `payments.xlsx` file.

Sender	Receiver	Amount
1309	1011	123051
1309	1011	118406
1309	1011	112456
1309	1011	120593
1309	1011	166396
1309	1011	177817
1309	1011	169351
1309	1011	181038

Figure 1: Snapshot of the input data structure in `Payments.xlsx`

4 Algorithm

The overall process employs a combination of graph processing, representation learning, and clustering algorithms.

4.1 Graph Construction

- **Input:** List of transactions (Sender, Receiver, Amount).
- **Process:** A directed graph (`networkx.DiGraph`) is created. Accounts are added as nodes. Transactions are iterated, and amounts are summed for identical (Sender, Receiver) pairs. These aggregated sums become the weights of the directed edges. Isolated nodes are subsequently removed.
- **Output:** A weighted, directed graph representing the transaction network.

4.2 Node2Vec Embedding Generation

This core algorithm learns node representations.

- **Input:** The constructed graph, walk parameters (number of walks per node, walk length, return parameter ‘p’, in-out parameter ‘q’), embedding dimension size, and training hyperparameters.
- **Sub-process 1: Biased Random Walks:**
 - For each node, multiple random walks are simulated. Each walk starts at the node and proceeds for a fixed length.
 - The transition probability between nodes depends on edge weights and the ‘p’ and ‘q’ parameters, which control the balance between exploring locally (Breadth-First Search style) and exploring further away (Depth-First Search style). In this specific execution, ‘p=1.0’ and ‘q=1.0’ were used, making the walks primarily influenced by edge weights (akin to DeepWalk).
 - **Alias Sampling:** An efficient technique ($O(1)$ complexity) is employed to sample the next node during a walk according to the potentially non-uniform transition probabilities derived from weights and p/q parameters. Transition probabilities and alias tables are precomputed for efficiency.
- **Sub-process 2: Skip-gram with Negative Sampling (SGNS):**
 - **Model Architecture:** A shallow neural network (`NodeEmbeddingModel` implemented using PyTorch `nn.Embedding`) is used. It typically consists of two embedding layers: one for the target node and one for the context nodes.
 - **Learning Objective:** The generated walks serve as input sequences. The model is trained to predict the context nodes (nodes appearing nearby in a walk) given a target node. This is achieved using the Skip-gram objective, optimized via Negative Sampling. The model aims to maximize the similarity (dot product of embeddings) between the target node and its true context nodes while minimizing the similarity with randomly sampled ‘negative’ nodes (nodes not in the context).
 - **Training Procedure:** The model is trained using an optimizer like Adam over multiple epochs, processing the walk data in batches.
- **Output:** A matrix of node embeddings, where each row corresponds to a node and contains its learned low-dimensional vector representation (dimension 64 in this run).

4.3 K-Means Clustering

- **Input:** The node embedding matrix generated by Node2Vec, and the desired number of clusters (K).
- **Process:** The standard K-Means algorithm is applied to the node embeddings. It partitions the N nodes (represented by their embedding vectors) into K clusters. The objective is to minimize the Within-Cluster Sum of Squares (WCSS), which measures the sum of squared Euclidean distances between each node’s embedding and the centroid of its assigned cluster. The implementation uses ‘`sklearn.cluster.KMeans`’ with ‘`n_init=10`’ runs with different centroid seeds to improve the quality and stability of the result.
- **Output:** An array containing the cluster assignment label (an integer from 0 to K-1) for each node.

4.4 Optimal K Selection (Elbow Method)

- **Input:** Node embeddings and a predefined range of candidate K values (e.g., 2 to 6).
- **Process:** K-Means clustering is performed iteratively for each value of K within the specified range. For each K, the resulting WCSS (inertia) is recorded. The WCSS values are then typically plotted against K. The ‘elbow’ point on this plot, where the rate of decrease in WCSS sharply diminishes, is considered indicative of a suitable number of clusters. This specific implementation identifies the elbow by finding the K value (within the tested range, excluding endpoints) that has the maximum perpendicular distance to the line segment connecting the WCSS points of the minimum and maximum K values tested.
- **Output:** An estimated optimal number of clusters (K).

4.5 Dimensionality Reduction (PCA)

- **Input:** The high-dimensional node embedding matrix (e.g., 799 nodes x 64 dimensions).
- **Process:** Principal Component Analysis (PCA), implemented via `'sklearn.decomposition.PCA'`, is used to reduce the dimensionality of the embeddings. It projects the data onto a lower-dimensional subspace (typically 2D for visualization) spanned by the principal components, which capture the directions of maximum variance in the data.
- **Output:** A lower-dimensional representation of the node embeddings (e.g., 799 nodes x 2 dimensions).

5 Approach

The script implements the algorithms described above in a sequential pipeline using Python, leveraging libraries like Pandas, NetworkX, PyTorch, Scikit-learn, and Matplotlib.

5.1 Data Preparation

- The `'DataProcessor'` class handles loading the `payments.xlsx` file.
- It performs necessary checks for column existence, converts data types (Sender/Receiver to string, Amount to numeric), and handles missing values by dropping affected rows.
- Console Output [1/6] confirms this step, reporting 130,535 initial transactions loaded.

5.2 Graph Construction Implementation

- The `'GraphCreator'` class takes the processed DataFrame.
- It uses a `'defaultdict(float)'` to efficiently aggregate transaction amounts between identical sender-receiver pairs.
- A `'networkx.DiGraph'` object is populated with unique accounts as nodes and aggregated transaction totals as weighted directed edges.
- Isolated nodes are identified using `'nx.isolates'` and removed.
- Mappings between string node IDs and integer indices (required by PyTorch Embedding layers) are created.
- Basic graph validity checks are performed (e.g., sufficient nodes for clustering range).
- Console Output [2/6] summarizes the resulting graph: 799 nodes, 5358 edges.

5.3 Node2Vec Implementation Details

- **Walk Generation:**
 - Parameters: 15 walks per node, walk length 40, `'p=1.0'`, `'q=1.0'`.
 - The `'TransitionCalculator'` and `'AliasMethod'` classes are used within `'precompute_transition_probs'` to efficiently calculate and store transition probabilities and alias tables for sampling.
 - The `'simulate_single_walk'` function generates individual walks using the precomputed tables.
 - The generated walks (lists of node IDs) are converted to index-based walks via `'convert_walks_to_indices'`.
 - Console Output [3/6] (part 1) notes the generation of 10,545 valid walks.
- **Embedding Training:**
 - The `'EmbeddingTrainer'` class encapsulates the training logic. It initializes the `'NodeEmbedding-Model'` (a PyTorch `'nn.Module'` with two `'nn.Embedding'` layers) and the Adam optimizer.
 - Training parameters: 5 epochs, learning rate 0.01, window size 5, batch size 128, 5 negative samples per positive sample.

- The ‘BatchGenerator’ class prepares mini-batches of (target, context, negative samples) from the indexed walks.
- The training loop iterates through epochs and batches, computing the negative sampling loss and updating model weights.
- Console Output [3/6] (part 2) provides epoch-wise loss and timing, showing training convergence.
- The final embeddings are extracted from the trained ‘model.node_embeddings.weight’.

5.4 Clustering Execution

- The ‘ClusteringAnalyzer’ class orchestrates the clustering process.
- ‘find_optimal_k_elbow’ iterates through the specified K range ([2, 6]), running ‘KMeans’ for each K, recording WCSS (inertia), and applying the elbow heuristic (distance-to-line method) to estimate the best K.
- Console Output [4/6] reports the WCSS for K=2 through K=6 and the determined optimal K=4.
- ‘perform_final_clustering’ runs ‘KMeans’ one final time using the optimal K=4 with ‘n_init=10’ to obtain the definitive cluster assignments for all nodes.
- Console Output [5/6] signifies the execution of this final clustering step.

5.5 Visualization Generation

- The ‘EmbeddingVisualizer’ class handles the final visualization steps.
- ‘reduce_dimensions_pca’ applies PCA to project the 64D embeddings down to 2D.
- ‘plot_and_save_clustered_embeddings’ generates a scatter plot using Matplotlib:
 - The x and y axes represent the first two principal components.
 - Each point corresponds to a node.
 - Points are colored according to the final cluster assignments (from K=4 clustering).
 - The plot includes informative labels, title (with K, node/edge counts), explained variance percentages for the axes, a legend, and grid lines.
- The generated plot is saved as a PNG file (`clustered_embeddings_pca.png`) in the specified output directory (`clustering_plots`).
- Console Output [6/6] confirms the PCA reduction and plot saving.

6 Results and Analysis

6.1 Summary of Outputs

The execution pipeline produced the following key quantitative results:

- **Processed Graph:** A directed graph with 799 nodes and 5,358 weighted edges was constructed from 130,535 transactions.
- **Node Embeddings:** 64-dimensional vector embeddings were generated for each of the 799 nodes using Node2Vec (p=1, q=1) trained for 5 epochs. The final average training loss was approximately 0.5084.
- **Optimal Cluster Count (K):** The Elbow method analysis over the range K=[2, 6] suggested K=4 as the optimal number of clusters based on the WCSS values:
 - K=2: 41509.18
 - K=3: 40569.02
 - K=4: 40003.71

- K=5: 39573.80
- K=6: 39266.72

The heuristic identified K=4 as the point balancing WCSS reduction and model complexity.

- **Final Clustering:** K-Means algorithm assigned each node to one of the 4 determined clusters.
- **PCA Dimensionality Reduction:** The 64D embeddings were reduced to 2D using PCA. The first principal component explained 7.8% of the variance, and the second explained 3.5%, for a total of 11.3% explained variance in the 2D representation.

6.2 Clustering Results and Visualization

The final cluster assignments were visualized using a scatter plot of the 2D PCA-reduced embeddings, as shown in Figure 2.

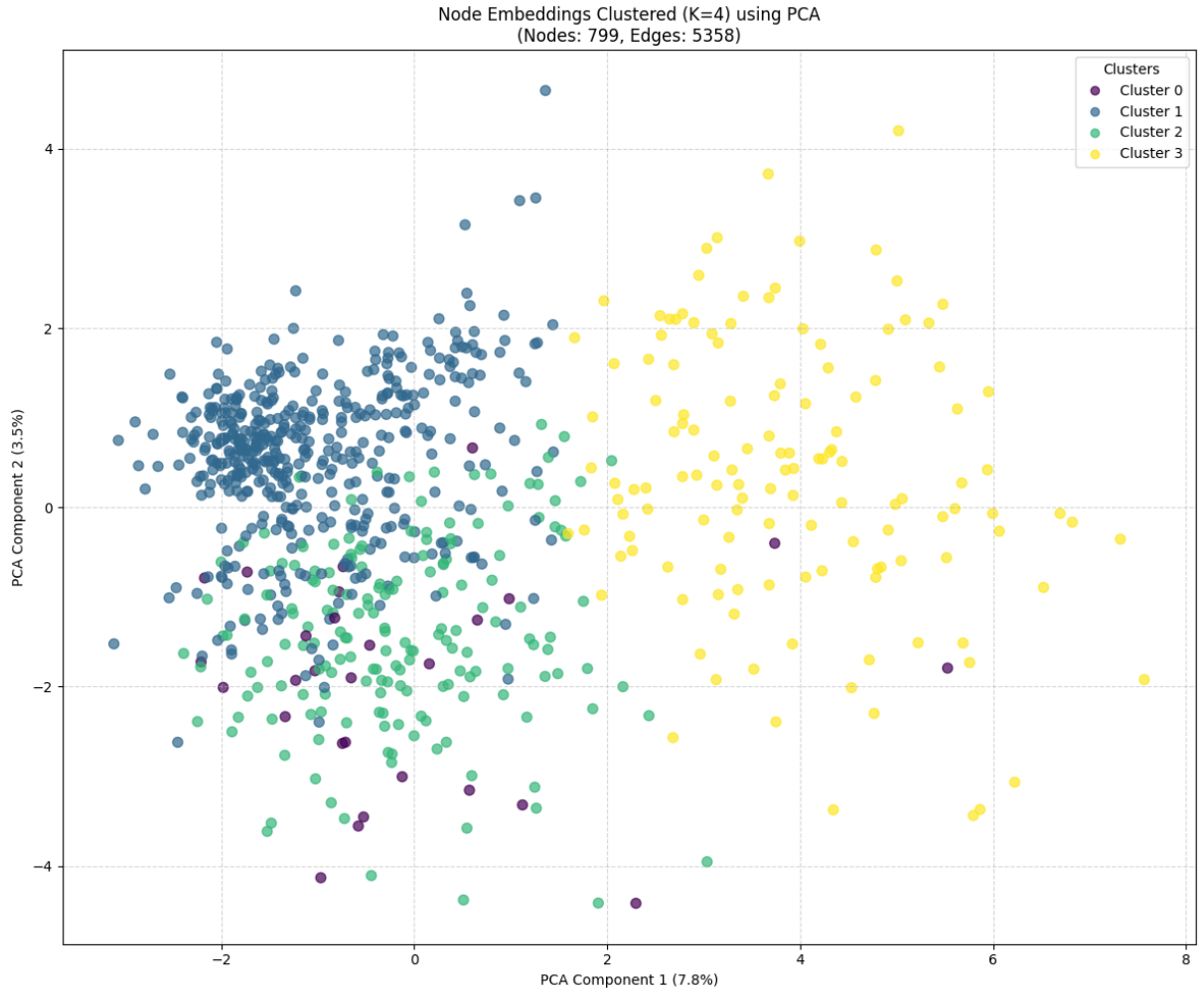


Figure 2: PCA Visualization of Node Embeddings Clustered with K=4

6.3 Analysis and Interpretation

- **Effectiveness of Embedding:** The Node2Vec algorithm successfully generated embeddings that allowed K-Means to partition the nodes. The convergence during training suggests the model learned meaningful representations capturing some aspects of the graph structure.
- **Cluster Structure in Visualization:** The PCA plot (Figure 2) provides a glimpse into the cluster structure.
 - Cluster 3 (yellow points) appears somewhat separated from the others, primarily along the horizontal axis (PCA Component 1). This might indicate a group of accounts with distinct connectivity patterns compared to the rest of the network, as captured by the first principal component.

- Clusters 0 (dark blue/purple), 1 (purple), and 2 (green/teal) exhibit significant overlap in this 2D projection. This could mean several things: (a) these communities are genuinely structurally similar or interconnected in the graph, (b) the chosen $K=4$ forced a separation where a continuum might exist, or (c) the primary distinguishing features between these clusters lie in dimensions not well captured by the first two principal components.
- **Impact of Low Explained Variance:** The very low total explained variance (11.3%) by the two PCA components is a critical point. It strongly suggests that the 2D visualization is a highly simplified representation of the actual relationships between node embeddings in the 64-dimensional space. Conclusions drawn solely from the visual separation (or lack thereof) in this plot should be made with caution. Clusters appearing overlapped here might be clearly distinct in the original embedding space.
- **Overall Insight:** Despite the visualization limitations, the process demonstrates that applying graph embedding techniques like Node2Vec followed by clustering can uncover latent groupings within complex transaction networks. The identification of 4 clusters suggests the presence of different interaction patterns or communities among the accounts.

6.4 Limitations and Future Work

- **Node2Vec Parameters:** The use of ‘ $p=1$, $q=1$ ’ makes the walk generation similar to DeepWalk, emphasizing local neighborhood similarity (homophily). Exploring different ‘ p ’ and ‘ q ’ values could potentially reveal different types of structural equivalence or roles within the network.
- **PCA Limitation:** As noted, the low variance captured by 2D PCA limits the interpretability of the visualization. Using techniques like t-SNE or UMAP might provide alternative visualizations, although they also come with their own interpretation challenges. Analyzing cluster separation directly in the 64D space (e.g., using silhouette scores) would provide a more robust measure of cluster quality.
- **Elbow Method Subjectivity:** The Elbow method is a heuristic and can sometimes be ambiguous. Evaluating different K values using other metrics (like silhouette score or Davies-Bouldin index) or incorporating domain knowledge could help confirm or refine the choice of $K=4$.
- **Static Analysis:** The analysis considers the graph as static, based on aggregated transactions over the entire dataset period. Analyzing temporal graph snapshots or using dynamic graph embedding techniques could reveal evolving community structures.

6.5 Conclusion

This assignment successfully implemented a pipeline to perform community detection in a transaction graph using Node2Vec embeddings and K-Means clustering. Four clusters were identified based on the Elbow method heuristic applied to the learned node representations. While the 2D PCA visualization offered limited insight due to low explained variance, the overall process demonstrates the potential of graph machine learning techniques to uncover structural patterns in financial networks, which could be valuable for downstream tasks like anomaly detection or understanding network dynamics.